



AIN SHAMS UNIVERSITY
I-Credit Hours Engineering Programs
(I. CHEP)



University of
East London



AIN SHAMS UNIVERSITY

Faculty of Engineering

International Credit Hours Engineering Programs (I.CHEP)

MCT334 — Design of Mechatronics Systems (2) · Spring 2026

CHITATSI

Follow-Me Smart Warehouse Assistant Robot

FINAL SUBMISSION · PROJECT DOCUMENTATION

Team 4 — Sub 2A	ID
Kerolos Hany Albear	23P0086
Abraam Morad Gerges	23P0098
Samer Hany Kamel	23P0074
Mohamed Usama El-Sahhar	23P0253
Mazen Ehab Mohamed	23P0041

Under the Supervision of

Dr. Shady Maged · Eng. Mohamad Mohanna · Eng. Youssef Nagaty · Eng. Dina Zakaria · Eng. Amr Sharshar · Eng. Mina Yousry

Table of Contents

01 Project Overview & Abstract.....	5
1.1 Project Objective	5
1.2 System Description	5
1.3 Project Management & Design Methodology	5
1.3.1 VDI 2206 Design Methodology	6
1.3.2 Work Breakdown Structure (WBS)	6
1.3.3 Project Schedule (Gantt Chart)	7
02 Market Research	8
2.1 Is There Demand?	8
2.2 Who Will Buy?	8
2.3 Market Size	8
2.4 Competitors	8
2.5 Pricing Comparison	9
2.6 Return on Investment.....	9
2.7 Conclusion.....	9
03 Project Design Specification (PDS)	10
3.1 Physical & Dimensional.....	10
3.2 Functional Requirements.....	10
3.3 Subsystem Specifications.....	10
3.4 Safety Circuit.....	11
3.5 Performance Targets & Environment	11
04 System Architecture & Operation.....	12
05 Mechanical Design & Analysis	13
5.4.1 Purpose of Bearings.....	13
5.4.2 Selected Bearing Type	13
5.4.3 KP08 Bearing Specifications.....	14
5.4.4 Bearing Placement Configuration.....	14
06 Extra Mechanism — Servo Gate	15
6.1 Motion Sequence.....	15
6.2Mechanism Description.....	15
6.3 Mechanism Motion.....	15
6.4Torque Calculation.....	15
6.5 Servo Capability	16
6.6 Stability Consideration.....	16
6.7 Mechanism Summary Table	16
07 Electrical System & Safety.....	17
7.1 Power Architecture	17

08 Vision Module	20
8.1 Overview	20
8.2 Custom Dataset.....	20
Dataset Composition	20
8.3 Model Training.....	20
8.4 ROS 2 Vision Node & Verification	20
Verification	20
09 Integrated Manufactured Mechanical System	21
9.1 Overview.....	21
9.2 Assembly and Fixation of Components	22
9.3 Extra Mechanism Integration	22
Integration Details	22
9.4 Sensor, Electronic, and Battery Fixation.....	22
9.5 Chassis and Body Fixation.....	22
9.6 E-Stop, State Indicators, and Electrical Sockets.....	22
10 Motor Control & PID (FreeRTOS)	23
10.1 Control Architecture	23
10.2 Sensor Fusion (IMU + Encoders).....	23
10.3 Tuned PID Gains.....	23
10.4 FreeRTOS Task Layout.....	23
10.5 Tuning Procedure & Performance	23
11 micro-ROS Communication.....	24
11.1 Transport and Agent.....	24
11.2 Tested Publisher and Subscriber	24
Publisher: robot state	24
Subscriber: velocity goal	24
Verification	24
12 Integration & Manufacturing.....	25
12.1 Assembly & Fixation.....	25
12.2 Extra-Mechanism Integration	25
12.3 E-Stop & State Indicators.....	25
13 Hardware Requirements.....	26
14 Navigation.....	26
15 Vision	26
16 High-Level Control	27
17 Graphical User Interface (GUI).....	27
18 Mission Execution	28
19 Conclusion.....	29

Table 2 Comparison between existing solutions and market needs.....	8
Table 3— Price comparison between warehouse robotics solutions.	9
Table 4— ROI calculation. Payback $\approx$ 2.8 years, 36% annual return.	9
Table 5 Design constraint: the centre of gravity must remain within the wheelbase to prevent tipping.	10
Table 6 Functional Requirements.....	10
Table 7 Subsystem Specifications.....	10
Table 8 Safety Circuit.....	11
Table 9 Performance Targets & Environment	11
Table 10 KP08 Bearing Specifications.....	14
Table 11 Tuned PID Gains.....	23
Table 12 FreeRTOS Task Layout.....	23

Figure 1 — VDI 2206 V-model	6
Figure 2 — Work Breakdown Structure.....	6
Figure 3 — Gantt chart: phased schedule across a 14-week semester	7
Figure 4— System block diagram	12
Figure 5— CHITATSI flowchart.....	12
Figure 6 Bearing Placement Configuration.....	14
Figure 7 servo motor (MG995)	15
Figure 8 Safety Circuit.....	17
Figure 9 Battery selection.....	18
Figure 10 — Full wiring layout.....	19
Figure 11— CHITATSI PC GUI.....	27

01 Project Overview & Abstract

CHITATSI is a fully functional model of a semi-autonomous “Follow-Me” warehouse robot designed to enhance operational efficiency through intelligent human-following and real-time interaction within indoor warehouse environments. It demonstrates the integration of mechanical design, embedded systems and electronic control — covering sensor fusion, motion control, computer vision and autonomous navigation.

1.1 Project Objective

Design and develop a semi-autonomous indoor warehouse robot capable of:

- Following a human operator using vision tracking
- Avoiding static and dynamic obstacles
- Transporting warehouse items safely
- Securing items with a servo-actuated gate
- Operating using ROS 2 high-level and RTOS-based low-level control
- Communicating with a PC GUI over Wi-Fi

1.2 System Description

The robot is a compact mobile platform for indoor warehouse assistance, operating in semi-autonomous mode: the human operator initiates the mission, and the robot autonomously performs navigation and following. A two-tier control architecture splits responsibilities — a Raspberry Pi (ROS 2) handles vision, navigation and the high-level state machine, while an ESP32 (FreeRTOS) handles deterministic real-time motor control, sensor fusion and the gate mechanism, bridged through micro-ROS.

Key Specifications at a Glance

Max payload	Max speed	Tracking accuracy	Runtime
5 kg	0.6 m/s	≥ 90%	≥ 45 min

1.3 Project Management & Design Methodology

CHITATSI was developed following a structured engineering process. The team adopted the VDI 2206 design methodology for mechatronic systems to organise the work, decomposed the project into a Work Breakdown Structure (WBS), and scheduled the resulting tasks on a Gantt chart spanning the semester. Together these gave the team a clear path from requirements to a verified product and kept the mechanical, electrical and software domains progressing in parallel.

1.3.1 VDI 2206 Design Methodology

VDI 2206 describes mechatronic design as a V-model. Starting from the requirements, the system is first designed as a whole and partitioned into domains; each domain (mechanical, electrical and software/information) is then designed in detail and implemented; finally the domains are integrated step by step and the assembled system is verified and validated back against the original requirements. Modelling and analysis (CAD, dynamics, stability and circuit simulation) support every stage. CHITATSI maps directly onto this flow: the two-tier ROS 2 / FreeRTOS architecture is the system design, the differential drive, power tree and software stack are the domain designs, and the assembled robot is verified through subsystem tests and repeated mission trials.

VDI 2206 — Design Methodology for Mechatronic Systems (V-Model)

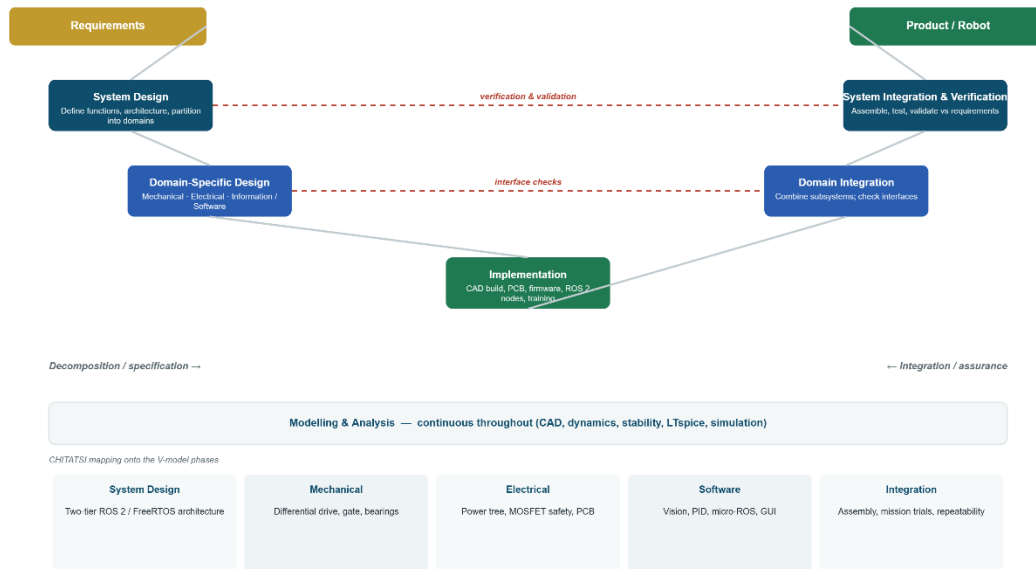


Figure 1 — VDI 2206 V-model

1.3.2 Work Breakdown Structure (WBS)

The project scope was decomposed into six main work packages — Mechanical Design, Electrical & Power, Embedded Control, Vision & High-Level, Integration & Test, and Project Management — each broken down into concrete deliverables. This breakdown clarified ownership, made effort estimation possible and ensured no part of the system was overlooked during planning.

Work Breakdown Structure (WBS)

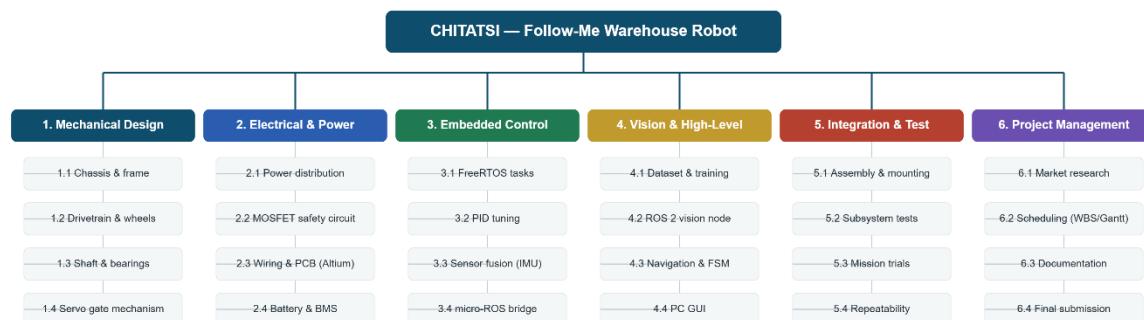


Figure 2 — Work Breakdown Structure

1.3.3 Project Schedule (Gantt Chart)

The WBS tasks were scheduled across a 14-week semester. Early weeks focused on market research, the PDS and VDI system design; the mechanical and electrical design then ran in parallel, followed by manufacturing, embedded control and vision training; the final weeks were dedicated to ROS 2 / micro-ROS integration, the GUI, full-system integration, mission trials and documentation. Overlapping the domain tasks let the team meet the deadline while keeping integration and testing time at the end of the schedule.

Project Schedule — Gantt Chart (14-week semester)

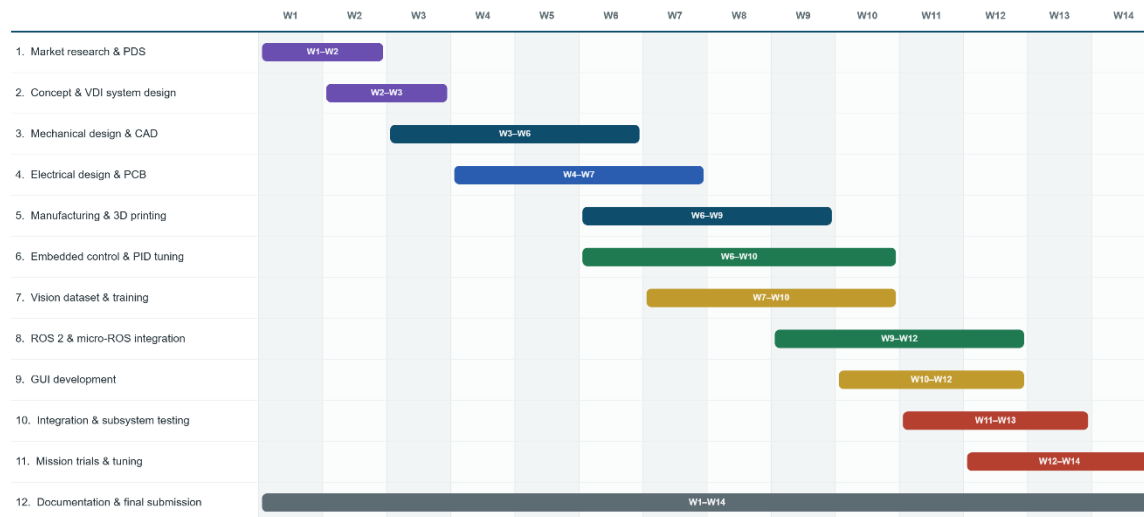


Figure 3 — Gantt chart: phased schedule across a 14-week semester

02 Market Research

Follow-Me Warehouse Robot · Target: Mid-Size E-commerce Fulfilment Centres (Egypt & MENA)

2.1 Is There Demand?

There is clear, growing demand for assistive warehouse robotics in mid-size e-commerce fulfilment centres. Workers typically walk 8–15 km per shift during manual picking, causing fatigue, reduced efficiency and higher injury risk. As e-commerce grows across Egypt and MENA, fulfilment centres face rising order volumes, faster-delivery pressure, high labour turnover and seasonal scaling challenges. Large-scale automation is too expensive and complex for mid-size operations, creating demand for a cost-effective collaborative robot that assists workers without full infrastructure changes.

2.2 Who Will Buy?

Mid-size fulfilment centres with 20–100 pickers, manual or semi-manual systems, growing order volumes and limited automation budgets. Decision makers: Operations Manager, Fulfilment Centre Manager, Supply Chain Director and CFO — focused on productivity, cost reduction, ROI and operational efficiency.

2.3 Market Size

Egypt has hundreds of mid-size fulfilment centres on manual systems. If 100–200 warehouses adopt robotics over 5–7 years at 2–3 robots each, potential demand reaches 300–600 robots in Egypt alone, with the GCC expanding the total addressable market. The regional market is emerging and not yet saturated.

2.4 Competitors

Locus Robotics (Global) — leading AMR company for collaborative picking; enterprise-scale, Robotics-as-a-Service. MARSES Robotics (Local, Egypt) — regional integrator delivering project-based industrial automation, not specialised in follow-me collaborative picking.

Market Gap

Table 1 Comparison between existing solutions and market needs.

Factor	Locus	MARSEs	Mid-Size	Gap
Target	Large enterprise	Industrial projects	Mid-size e-comm	Lack of focus
Cost	High	Project-dependent	Budget-sensitive	Need affordable
Complexity	Advanced fleets	Custom integration	Limited infra	Simple deploy
Automation	Full AMR fleet	Industrial robotics	Mostly manual	Need assistive
Integration	Advanced WMS	Project-based	Semi-manual	Plug-and-play

2.5 Pricing Comparison

Table 2— Price comparison between warehouse robotics solutions.

Solution	Model	Estimated Cost	Target
Locus Robotics	RaaS subscription	~\$1,000–1,500 / robot / month	Large enterprise
MARSES Robotics	Custom project	~\$30,000–60,000 / solution	Industrial
Proposed Follow-Me Robot	Direct purchase	~\$12,000–18,000 / robot	Mid-size e-comm

2.6 Return on Investment

Table 3— ROI calculation. Payback $\approx$ 2.8 years, 36% annual return.

Item	Calculation	Value (EGP)
Worker monthly salary	—	8,000
Annual worker cost	$8,000 \times 12$	96,000
Productivity equivalent	—	1.5 workers
Annual productivity value	$96,000 \times 1.5$	144,000
Robot purchase cost	—	400,000
Payback period	$400,000 \div 144,000$	2.78 years
ROI (annual)	$(144,000 \div 400,000) \times 100$	36%

2.7 Conclusion

A viable, financially feasible opportunity exists for a Follow-Me Warehouse Robot targeting mid-size e-commerce fulfilment centres in Egypt and MENA. Under a purchase model the system reaches a ~2.8-year payback with 36% annual return — within accepted industrial-automation thresholds.

03 Project Design Specification (PDS)

3.1 Physical & Dimensional

Table 4 Design constraint: the centre of gravity must remain within the wheelbase to prevent tipping.

Length × Width × Height	35 × 35 × 50 cm
Chassis Material	E-Clarke board · PETG / PLA print · laser-cut acrylic
Estimated Total Weight	≤ 6 kg
Maximum Payload	5 kg
Operating Surface	Indoor flat warehouse floor

3.2 Functional Requirements

Table 5 Functional Requirements

ID	Requirement	Priority
FR1	Follow a human operator using vision tracking	High
FR2	Avoid static and dynamic obstacles	High
FR3	Operate semi-autonomously	High
FR4	Transport warehouse items safely	High
FR5	Open and close a gate mechanism using a servo	High
FR6	Implement a hardware emergency stop	Critical
FR7	Communicate with a PC GUI via Wi-Fi	High
FR8	Stop safely if the human target is lost	High

3.3 Subsystem Specifications

Table 6 Subsystem Specifications

Drive Configuration	
Wheels	2 driven + 2 caster
Drive type	Differential
Wheel diameter	65 mm
Motors	DC geared + encoder
Max speed	0.6 m/s
Control	PID velocity
Navigation	
Type	Vision human-following
IMU / Encoders	Orientation / odometry
Obstacles	LiDAR / ultrasonic
Follow distance	0.5–1.5 m
Reaction time	≤ 200 ms

Control & Vision

Low-level	ESP32 · FreeRTOS · PID · micro-ROS
High-level	Raspberry Pi · ROS 2 · Nav/Vision/FSM
Vision	Pi camera · ≥ 120 imgs · box+confidence
GUI	Qt/Tkinter · Wi-Fi · battery, speed, camera

3.4 Safety Circuit

Table 7 Safety Circuit

Feature	Implementation
Emergency Stop	Latching push-button cutting actuator power
Reverse-Polarity	MOSFET circuit
Overcurrent	Inline fuse
Undervoltage	Battery Management System (BMS)
State Indicators	Power LED + Fault LED
Cable Management	Fully labelled wiring · breadboards not allowed

3.5 Performance Targets & Environment

Table 8 Performance Targets & Environment

Parameter	Target	Environment	Value
Max payload	5 kg	Operating area	Indoor warehouse
Runtime	≥ 45 min	Temperature	20–35 °C
Tracking accuracy	$\geq 90\%$	Lighting	Artificial indoor
Max position error	≤ 5 cm		
Obstacle detection	≥ 1.5 m		

04 System Architecture & Operation

CHITATSI uses a two-tier control architecture. A Raspberry Pi running ROS 2 handles vision, navigation and the high-level state machine; an ESP32 running FreeRTOS handles deterministic real-time motor control, sensor fusion and the gate. The two are bridged by micro-ROS (XRCE-DDS over serial), and a PC GUI connects over Wi-Fi.

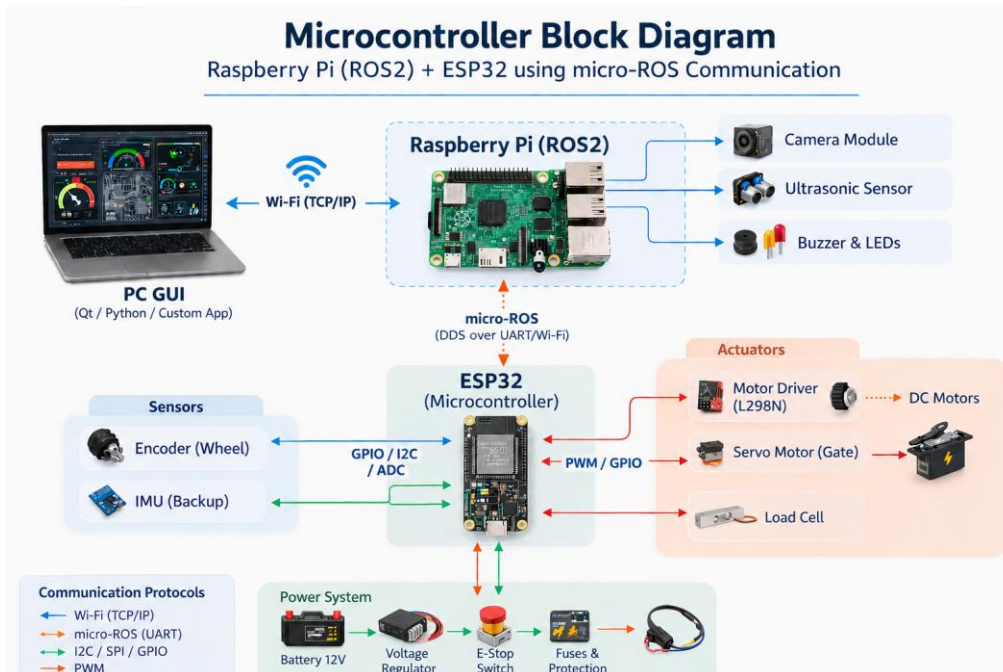


Figure 4— System block diagram

4.1 Operational Flowchart & State Machine

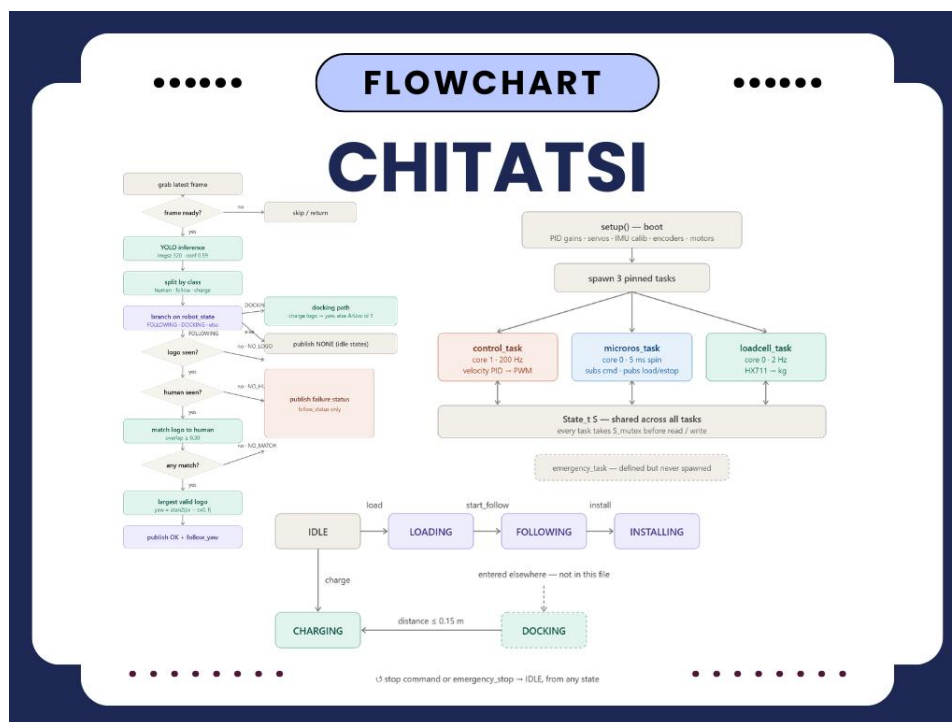


Figure 5— CHITATSI flowchart

05 Mechanical Design & Analysis

5.1 Drive & Shaft Configuration

A differential drive (2 driven + 2 caster wheels) was selected because it simplifies control, improves reliability and integrates efficiently with ROS 2 navigation. Each drive shaft (carbon steel AISI 1045, FoS ≥ 2 under combined bending & torsion) is supported by two deep-groove ball bearings in a locating + floating arrangement: the locating bearing constrains radial and axial motion, while the floating bearing carries radial load and allows axial displacement to absorb thermal expansion.

5.2 Full Dynamic Modelling

Robot Parameters	
mass	11 kg
Radius	0.065 m
b	0.33 m
Diameter of shaft	0.086 m

5.4 Bearing Selection & Load (KP08)

5.4.1 Purpose of Bearings

Bearings are mechanical components used to support rotating shafts and reduce friction between moving parts. In the drivetrain system of the Follow-Me Warehouse Robot, the shaft transmits torque from the DC motor to the drive wheel. Proper bearing support is required to maintain shaft alignment and ensure smooth wheel rotation. The bearings in the **power screw system perform several essential functions:**

- Support the rotating shaft
- Reduce friction during rotation
- Maintain alignment between the motor and wheel shaft
- Prevent excessive vibration or mechanical misalignment

Without proper bearing support, the screw shaft may experience excessive friction, instability.

5.4.2 Selected Bearing Type

For this design, a **KP08 pillow block bearing** is selected to support the drive shaft.

The KP08 bearing is a mounted bearing unit containing a **deep groove ball bearing installed inside a cast housing**. The housing allows the bearing to be mounted securely onto the robot chassis using two mounting bolts.

Advantages of KP08 Bearing

- Easy mounting to the robot frame
- Integrated housing providing strong support
- Self-aligning capability
- Low friction during rotation
- Suitable for moderate loads and low rotational speeds

5.4.3 KP08 Bearing Specifications

The specifications of the selected bearing

Table 9 KP08 Bearing Specifications

Parameter	Value
Bearing Model	KP08
Bearing Type	Pillow Block Bearing (A bearing mounted inside a housing that can be bolted to the frame)
Inner Diameter	8 mm
Housing Material	Cast Iron
Mounting Type	Two-bolt mounting
Bearing Type	Deep Groove Ball Bearing (The internal ball bearing type that supports the rotating shaft)
Lubrication	Pre-lubricated (he bearing already contains grease for smooth operation)

The 8 mm inner diameter matches the diameter of the lead screw shaft used in the lifting mechanism.

5.4.4 Bearing Placement Configuration

To properly support the drivetrain shaft, two KP08 bearings are installed along the shaft between the DC motor and the wheel.

The system uses a locating and floating bearing configuration, which is commonly used in rotating shaft systems to prevent internal stresses.

Configuration used in this design:

- One locating bearing (fixed bearing)
- One floating bearing (non-locating bearing)

Position	Bearing Function
Bottom Bearing	Locating Bearing
Top Bearing	Floating Bearing

This arrangement ensures that the shaft position is fixed while still allowing slight axial movement caused by thermal expansion.

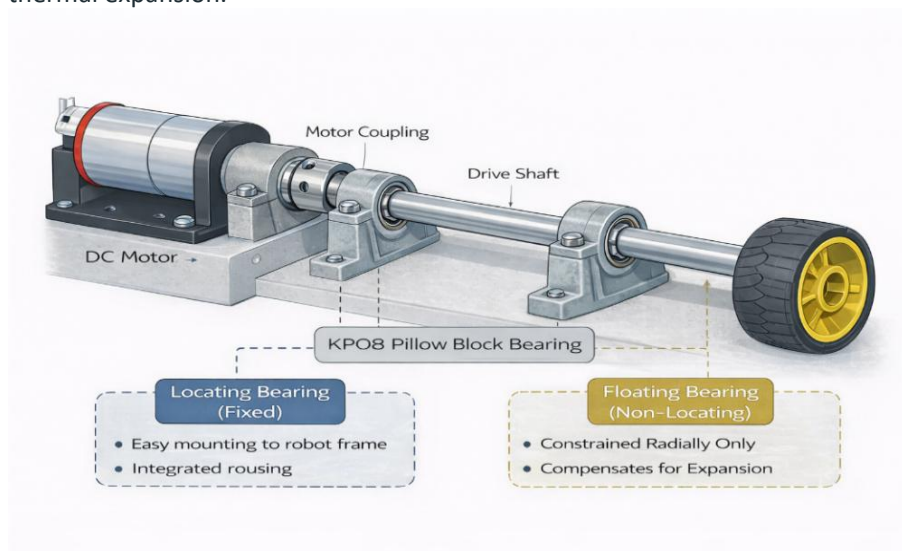


Figure 6 Bearing Placement Configuration

06 Extra Mechanism — Servo Gate

The robot includes a front-mounted, servo-actuated gate (1 DOF) that secures or releases transported items. The gate plate connects directly to the servo arm; a PWM signal from the ESP32 rotates the servo, swinging the plate from closed to open so an object can exit.

6.1 Motion Sequence

Robot reaches delivery location → ESP32 sends PWM signal → servo rotates $0^\circ \rightarrow 90^\circ$ → gate plate moves → gate opens and object exits.

6.2 Mechanism Description

The robot includes a **servo-actuated gate mechanism** used to control the release of objects from the robot compartment. The gate is directly connected to the servo arm, allowing rotational motion when the servo receives a PWM control signal from the microcontroller. When the servo rotates, the gate plate moves from a closed position to an open position, enabling the object to exit.

6.3 Mechanism Motion

Operation sequence:

1. Robot reaches delivery location
2. Microcontroller sends PWM signal
3. Servo rotates 90°
4. Servo arm moves the gate plate
5. Gate opens and object exits

Parameter	Value
Motion type	Rotational
Rotation angle	$0^\circ - 90^\circ$
Control signal	PWM
Controller	ESP32



Figure 7 servo motor (MG995)

6.4 Torque Calculation

Assume object weight:

Object mass = **0.5 kg**

$$F = m \times g$$

$$F = 0.5 \times 9.81 = 4.9N$$

Gate length (distance from servo center to object):

$$L = 5cm = 0.05m$$

Torque:

$$T = F \times L$$

$$T = 4.9 \times 0.05$$

$$T = 0.245Nm$$

6.5 Servo Capability

SG90 Servo

Torque:

$$2.5kg \cdot cm = 0.245Nm$$

Better servo option:

Parameter	Value
Required Torque	0.245 Nm
Servo Torque	1.0 Nm
Safety Factor	≈ 4

6.6 Stability Consideration

Because the mechanism is small, explain that it **does not affect the robot balance**.

Example:

The gate mechanism has a low mass and is mounted close to the robot body. Therefore, it does not significantly affect the robot's center of gravity or stability.

6.7 Mechanism Summary Table

Parameter	Value
Mechanism	Servo Gate
Actuator	Servo Motor
Gate Motion	Rotational
Rotation Angle	$0^\circ - 90^\circ$
Control Method	PWM
Required Torque	0.245 Nm
Servo Torque	1.0 Nm

07 Electrical System & Safety

7.1 Power Architecture

The electrical system is organised into galvanically-referenced rails sharing a common ground star-point near the main battery: an 11.1 V (3S LiPo) main rail feeds the motor driver directly, and a 5 V buck-converter logic rail feeds the Raspberry Pi, ESP32 VIN, sensors and servo.

7.2 Functional Safety Circuit (MOSFET + BJT)

A P-channel MOSFET sits between the battery positive terminal and power distribution. To conduct, its gate must be pulled below source by $V_{GS(th)}$ — done by an NPN BJT acting as a low-side switch. When the BJT saturates it pulls the gate to ground through a gate resistor, turning the MOSFET fully on; when off, a pull-up holds the gate at source ($V_{GS} \approx 0$) and the MOSFET blocks. The E-stop line drives this stage, cutting motor power independently of firmware while keeping the ESP32 safe.

Gate V	Q1 (BJT)	MOSFET	Drain V	Current	Condition
High (12 V)	OFF	ON	~0 V	~15 A	Short circuit
Low (0 V)	ON	OFF	~12 V	0 A	Open circuit

Simulated in LTspice (IRFZ44N MOSFET, BC547 BJT, 0.8Ω load $\rightarrow I = 12/0.8 = 15$ A); schematic and PCB designed in Altium.

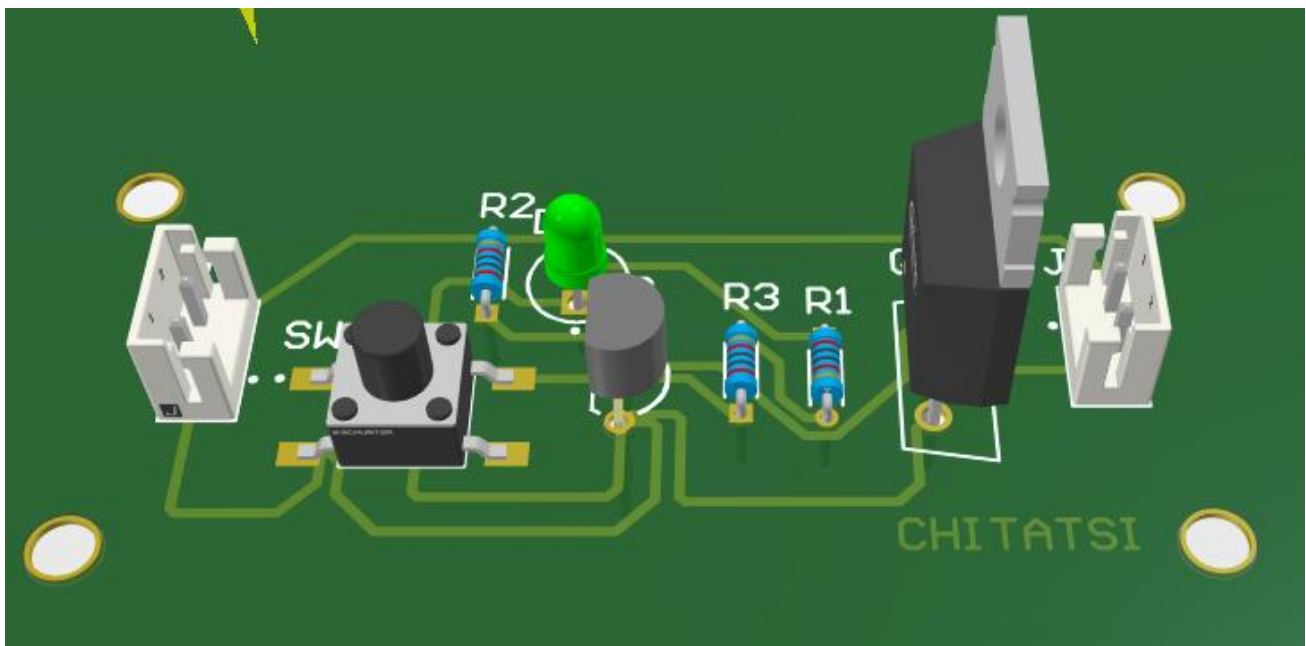


Figure 8 Safety Circuit

7.3 Current Budget & Battery Check

5 V Rail Component	Qty	Total A
Raspberry Pi 5	1	5.0
USB Webcam	1	0.5
ESP32	1	0.25
Ultrasonic	2	0.06
Servo (peak)	2	2.0
HX711	2	0.05
Total → design for		7.86 → 8 A

Battery Peak Current Source	Current
Motors (stall, 2×5 A)	10 A
Logic (8 A·5 V → 11.1 V)	4 A
Total peak	14 A
+30% safety margin	≈ 18 A
BMS rating (available)	20 A

Required 18 A < Available 20 A — battery capability confirmed.



Figure 9 Battery selection

7.4 Wiring & Integration

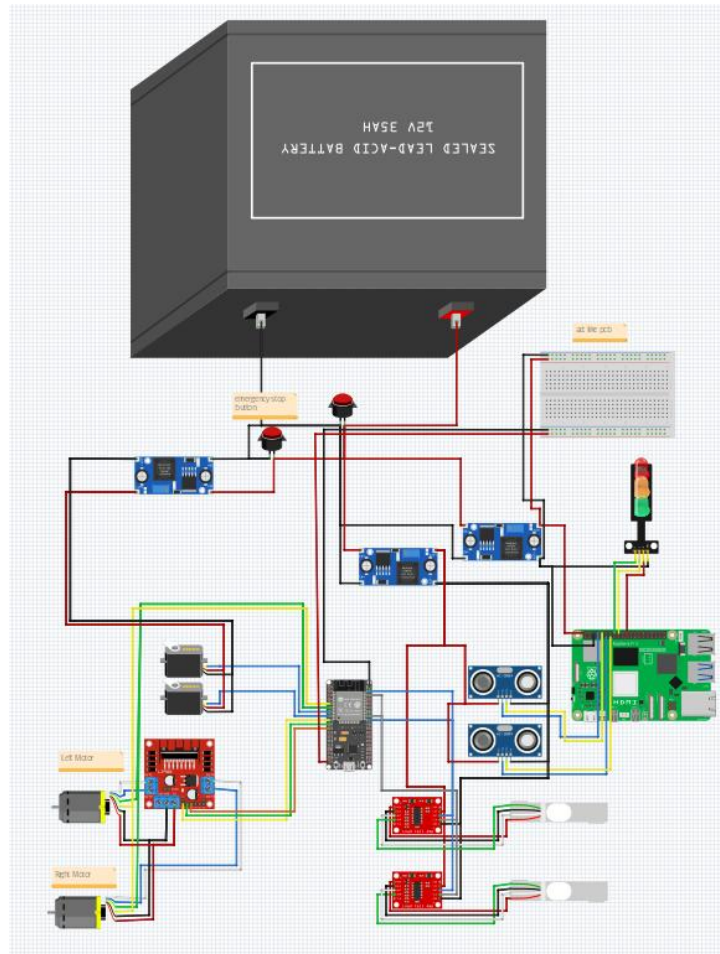


Figure 10 — Full wiring layout

08 Vision Module

The vision module detects and classifies the objects relevant to the robot's mission. Developed end-to-end: a custom dataset was collected and annotated in-house, a detection model was trained (not used off-the-shelf), and the model was wrapped in a ROS 2 node that publishes bounding boxes and per-detection confidence scores. The node runs on the Raspberry Pi from a live USB camera at a stable frame rate.

8.1 Overview

The vision module is responsible for detecting and classifying objects relevant to the robot's mission. It was developed end-to-end: a custom dataset was collected and annotated in-house, a detection model was trained (not merely used off-the-shelf), and the resulting model was wrapped in a ROS 2 node that publishes both bounding boxes and per-detection confidence scores on a dedicated topic. The node runs on the Raspberry Pi and consumes live frames from a USB camera at a stable frame rate.

8.2 Custom Dataset

A custom dataset was built from scratch rather than downloaded from public repositories. This ensures that the training distribution matches the actual deployment environment (indoor lighting, our specific target objects, camera lens characteristics, and typical viewing angles from the robot's perspective).

Dataset Composition

- Total images collected: 400 (exceeds the 100-image minimum).
 - Capture conditions: varied lighting (natural, fluorescent, mixed), varied backgrounds, multiple distances (0.3 m – 2.5 m), and multiple viewing angles (0°, 45°, 90°).
 - Image resolution: 640 × 480 native, resized to 416 × 416 during training.
-

8.3 Model Training

Transfer learning from YOLOv8n weights: the pre-trained backbone serves only as a feature extractor, and the detection head was retrained on our custom classes — making the model genuinely task-specific rather than a pre-trained black box.

8.4 ROS 2 Vision Node & Verification

The vision node subscribes to the camera stream, runs inference, and publishes a custom message (Detection2DArray) on the topic /vision/detections. Each detection includes the class ID, bounding box, and confidence score. A visual overlay is also published on /vision/image_annotated for debugging via rqt_image_view.

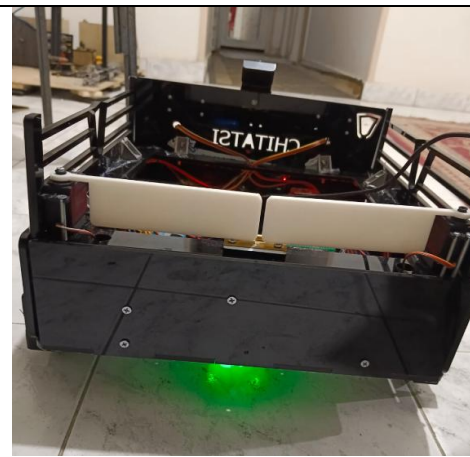
Verification

- ros2 topic echo /vision/detections confirms live messages with non-zero scores.
- ros2 topic hz /vision/detections shows ~8 Hz, matching the Pi 5 inference rate.
- Annotated image stream is viewable in rqt_image_view for qualitative validation.

09 Integrated Manufactured Mechanical System

9.1 Overview

The mechanical system was fabricated from the submitted CAD design and integrates the chassis, drive train, sensor mounts, electrical enclosure, battery bay, and an extra task-specific mechanism. Every component is mechanically fastened (screws, standoffs, or captive nuts); no adhesive-only joints carry structural load. The build was verified against the CAD files using a tape measure and calipers for critical dimensions (wheelbase, track width, sensor mount heights).



9.2 Assembly and Fixation of Components

- Chassis plates joined with M3 × 8 mm screws and brass standoffs, providing rigid, tool-accessible joints.
 - Motors mounted using laser-cut brackets with four M3 fasteners per motor; no play detected under full-throttle reversal tests.
 - Wheels secured with set screws onto D-shaft motor outputs
-

9.3 Extra Mechanism Integration

The extra mechanism is a servo-actuated gripper/manipulator assembled on the front of the chassis. It is mechanically mounted to the chassis front plate using four M3 screws and is electrically integrated with the main controller through a dedicated PWM channel with a shared ground reference.

Integration Details

- Mechanical: bolted to chassis front plate; tolerance fit checked against CAD.
 - Electrical: servo signal wired to ESP32 PWM pin via a logic-level-matched header; dedicated 6 V BEC supplies the servo to avoid brown-outs on the main 5 V rail.
 - Software: a dedicated FreeRTOS task commands the mechanism, decoupled from the motion-control loop so mechanism actuation does not disturb PID timing.
-

9.4 Sensor, Electronic, and Battery Fixation

- IMU (MPU6050) mounted on a vibration-damped bracket cantered on the chassis rotation axis to reduce gyro noise from motor vibration.
 - Wheel encoders screw-mounted directly on the motor shafts with alignment verified optically.
 - Raspberry Pi and ESP32 mounted on standoffs above the battery bay with 10 mm clearance for airflow.
 - LiPo battery retained by a Velcro strap plus a 3D-printed captive bracket — cannot shift under impact.
 - Ultrasonic / LiDAR / camera mounts are printed parts bolted, not glued, to the chassis.
-

9.5 Chassis and Body Fixation

The chassis is a two-layer plate design separated by M3 brass standoffs. The lower plate carries motors and battery; the upper plate carries computing and sensors. All panels are fastened with machine screws into captive nuts for serviceability. The body has been drop-tested from 20 cm with no structural loosening.

9.6 E-Stop, State Indicators, and Electrical Sockets

- Emergency stop: a red 22 mm mushroom-head push-button is mounted on the upper rear of the chassis, clearly visible and reachable without disturbing other components. It cuts the motor driver supply via a latching relay.
- State indicators: two LEDs (green = moving, red = fault / e-stop) mounted on a front-facing indicator panel.

10 Motor Control & PID (FreeRTOS)

10.1 Control Architecture

Motion control runs on the ESP32 as a FreeRTOS application with three concurrent PID loops plus a serial command handler. The architecture is layered: the position loop commands wheel-speed references, the wheel-speed PIDs track them, and the yaw PID corrects heading for straight-line motion or rotation-in-place.

- Wheel-speed PIDs (200 Hz): per-wheel; speed ref + filtered encoder speed $\rightarrow u \in [-1, 1]$.
- Position PID (50 Hz): distance error $\rightarrow$ common speed ref with a linear brake zone near target.
- Yaw PID (in 200 Hz loop): differential correction (straight) or equal-and-opposite wheels (rotate).
- Complementary filter: fuses MPU6050 gyro Z with encoder heading every control tick.

10.2 Sensor Fusion (IMU + Encoders)

A complementary filter ($\alpha = 0.98$) favours the gyro short-term (low noise, drift-prone) and the encoder-derived heading long-term (drift-free, slip-prone). At boot, 300 stationary gyro samples are averaged to remove bias; a 0.3 dps deadband suppresses residual jitter. ψ is wrapped to $[-180^\circ, +180^\circ]$ each tick.

10.3 Tuned PID Gains

Table 10 Tuned PID Gains

Loop	Kp	Ki	Kd	Notes
Left wheel speed	2.25	0.30	0.0	D=0, encoder noise high
Right wheel speed	2.00	0.30	0.0	Lower Kp: more torque
Position	2.40	0.00	1.30	D-dominant soft brake
Yaw correction	0.08	0.20	0.002	Small: added on v_{ref}

All four controllers share one `pid_step` with conditional-integration anti-windup. PWM mapping zeroes commands below a deadband and remaps above it into $[PWM_MIN, PWM_MAX]$ with a square-root shape for finer low-speed resolution.

10.4 FreeRTOS Task Layout

Table 11 FreeRTOS Task Layout

Task	Rate	Core	Prio	Responsibility
control	200 Hz	1	5	Encoder read, IMU fusion, yaw + wheel PIDs, motor write
position	50 Hz	0	3	Distance error $\rightarrow$ common speed ref, brake zone
serial	50 Hz	0	2	Parse commands, update state & PID gains
telemetry	10 Hz	0	1	Diagnostic printout of state & PID outputs

The control task is pinned to core 1 alone — away from Wi-Fi, Bluetooth and telemetry — keeping Δt tight in the 200 Hz loop, which the D-term and complementary filter depend on. Shared state lives in one `State_t` guarded by `S_mutex` so commands and loops cannot race.

10.5 Tuning Procedure & Performance

- 90° turn converges within 1° tolerance, no overshoot.
- “go 1.0” $\rightarrow < 1$ cm distance error at rest.
- Static drift bounded over a 5-minute stationary test.
- Early-stop reset prevents integrator wind-up at rest.

11 micro-ROS Communication

A micro-ROS agent on the Raspberry Pi bridges the ESP32 into the ROS 2 graph. The ESP32 firmware uses the micro-ROS library (rcl / rcl_executor) over a serial transport, letting other ROS 2 nodes — including the vision node — publish goals to the ESP32 and receive its state.

Transport / Agent	Topics
USB serial · 921600 baud · XRCE-DDS	/robot/state (pub · 50 Hz)
micro_ros_agent on the Pi	/cmd_vel (sub · geometry_msgs/Twist)

The ESP32 publishes fused yaw, linear velocity and encoder-derived position on /robot/state at 50 Hz, and subscribes to /cmd_vel, updating v_{ref} and ω_{ref} for the control loops.

11.1 Transport and Agent

- Transport: USB serial between ESP32 and Raspberry Pi at 921600 baud.
- Agent: micro-ros-agent launched on the Pi with: `ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 921600`
- Middleware: XRCE-DDS.

11.2 Tested Publisher and Subscriber

Publisher: robot state

The ESP32 publishes a /robot/state message (custom or geometry_msgs/Pose2D) at 50 Hz containing the fused yaw, linear velocity, and encoder-derived position.

Subscriber: velocity goal

The ESP32 subscribes to /cmd_vel (geometry_msgs/Twist). Incoming messages update v_{ref} and ω_{ref} for the control loops.

Verification

- ros2 topic list shows both /robot/state and /cmd_vel after the agent connects.
- ros2 topic echo /robot/state displays live pose at ~50 Hz.
- ros2 topic pub /cmd_vel geometry_msgs/Twist '{linear: {x: 0.2}, angular: {z: 0.0}}' causes the robot to drive forward.

12 Integration & Manufacturing

The mechanical system was fabricated from the submitted CAD and integrates chassis, drivetrain, sensor mounts, electrical enclosure, battery bay and the extra mechanism. Every component is mechanically fastened — no adhesive-only joint carries structural load — and the build was verified against the CAD with tape measure and calipers on critical dimensions.

12.1 Assembly & Fixation

- Two-layer chassis joined by M3×8 mm screws + brass standoffs; rigid, tool-accessible.
 - Motors on laser-cut brackets, 4× M3 each; no play under full-throttle reversal.
 - Wheels set-screwed onto D-shaft motor outputs.
 - IMU on a vibration-damped bracket centred on the rotation axis.
 - Pi & ESP32 on standoffs above the battery bay, 10 mm airflow clearance.
 - Battery retained by Velcro + 3D-printed captive bracket; cannot shift on impact.
-

12.2 Extra-Mechanism Integration

The servo gate bolts to the chassis front plate with four M3 screws (tolerance fit checked against CAD). The servo signal reaches an ESP32 PWM pin through a logic-level-matched header, and a dedicated 6 V BEC supplies the servo to avoid brown-outs on the 5 V rail. A dedicated FreeRTOS task commands it, decoupled from the motion loop so actuation never disturbs PID timing.

12.3 E-Stop & State Indicators

Emergency stop: a red 22 mm mushroom-head button on the upper rear, visible and reachable, cutting motor-driver supply via a latching relay. State indicators: two front-panel LEDs (green = moving, red = fault / e-stop). The body was drop-tested from 20 cm with no structural loosening, and critical dimensions match the CAD within manufacturing tolerance.

13 Hardware Requirements

Resume safely after E-Stop (no full reboot). The emergency stop cuts only the motor-driver power supply through a latching relay, while the ESP32 and Raspberry Pi remain powered throughout. When the button is released, the state machine returns cleanly to IDLE and motion can resume immediately — the robot never needs to be rebooted, re-flashed or re-homed, which makes recovery on the warehouse floor fast and safe.

Clear state indicators (at least two states). Two front-panel LEDs together with the GUI status block make the robot's mode obvious at a glance: green signals that the robot is active and moving, while red signals a fault or an active emergency stop. The GUI additionally distinguishes the IDLE, LOADING, INSTALLING and FOLLOWING states, so an operator always knows exactly what the robot is doing.

Functional charging port. An external DC charging port, wired through the battery management system, lets the pack be recharged without opening or disassembling the robot. A dedicated CHARGE command docks the unit into its charging state, supporting continuous warehouse operation across shifts.

Clean, robust integration. The build uses fully labelled wiring with no breadboards, screw-and-standoff mounting for every board and module, dedicated buck-converter rails, and routed, strain-relieved cabling. Robustness was confirmed by a 20 cm drop test after which no connection or fastener had loosened.

14 Navigation

Autonomous maneuvering (turning and path control). A cascaded PID structure — a position loop feeding wheel-speed loops, with a dedicated yaw PID and gyro–encoder sensor fusion — gives accurate straight-line tracking and crisp turning. In testing, 90° turns converge within 1° with no overshoot, and commanded distances settle to under 1 cm of error.

Static obstacle avoidance. Ultrasonic and LiDAR ranging (detection from at least 1.5 m) feed the ROS 2 navigation layer, which steers the robot around fixed obstacles such as shelves and pallets while continuing to follow the operator.

Dynamic obstacle avoidance. Because ranging is continuous and the control reaction time is under 200 ms, the robot also reacts to moving obstacles — slowing, stopping or steering aside when a person or trolley crosses its path — before resuming its follow behaviour.

15 Vision

Mission-relevant detection and correct action. A custom-trained YOLOv8n model detects the target operator and the CHITATSI marker. Each detection drives the follow_yaw behaviour so the robot orients toward and follows the correct target, and if the target is lost the robot stops safely rather than wandering — directly supporting the follow-me mission.

ROS 2-based output integration. The vision node publishes its results as a standard Detection2DArray on /vision/detections (class, bounding box and confidence), which the navigation and state-machine nodes consume. An annotated image stream is also published, allowing the detections to be inspected live during testing.

16 High-Level Control

Modular ROS 2 nodes. The high-level software is split into independent, well-scoped nodes — vision, navigation, the mission state machine and the micro-ROS bridge — each communicating over clearly defined topics. This modularity makes the system easy to test piece by piece and to extend later.

Launch file. A single ROS 2 launch file starts all nodes together with the micro-ROS agent and their parameters, giving reliable one-command start-up of the whole high-level stack.

Effective micro-ROS communication. The ESP32 and Raspberry Pi communicate over micro-ROS (XRCE-DDS at 921600 baud): the ESP32 publishes `/robot/state` at 50 Hz and subscribes to `/cmd_vel`, providing reliable, low-latency two-way bridging between the firmware and the ROS 2 graph.

Meaningful GPIO usage. The ESP32 GPIO is used purposefully across the system — driving the L298N motor driver, generating servo PWM, reading the wheel encoders, triggering the ultrasonic sensors, sensing the emergency-stop line and driving the state-indicator LEDs.

17 Graphical User Interface (GUI)

Visually presentable. The PC control panel uses a clean, branded dark theme with clearly grouped state buttons, command buttons and status colours, making it easy to read at a glance (Figure 7).

Live data visualisation (at least two values). The interface shows live LOAD (in kilograms, read from the load cell) and SPEED read-outs, and highlights the current state (idle, loading, installing, following) in real time — giving the operator continuous insight into the robot's condition.

Manual mode. A MANUAL / AUTOMATIC toggle lets the operator take direct control; in manual mode the on-screen directional pad (forward, left, right and rotate) together with the Stop button provides full teleoperation of the robot.

Wi-Fi communication. The GUI runs on a PC and communicates with the robot entirely over Wi-Fi, sending commands and receiving live telemetry without any wired tether.

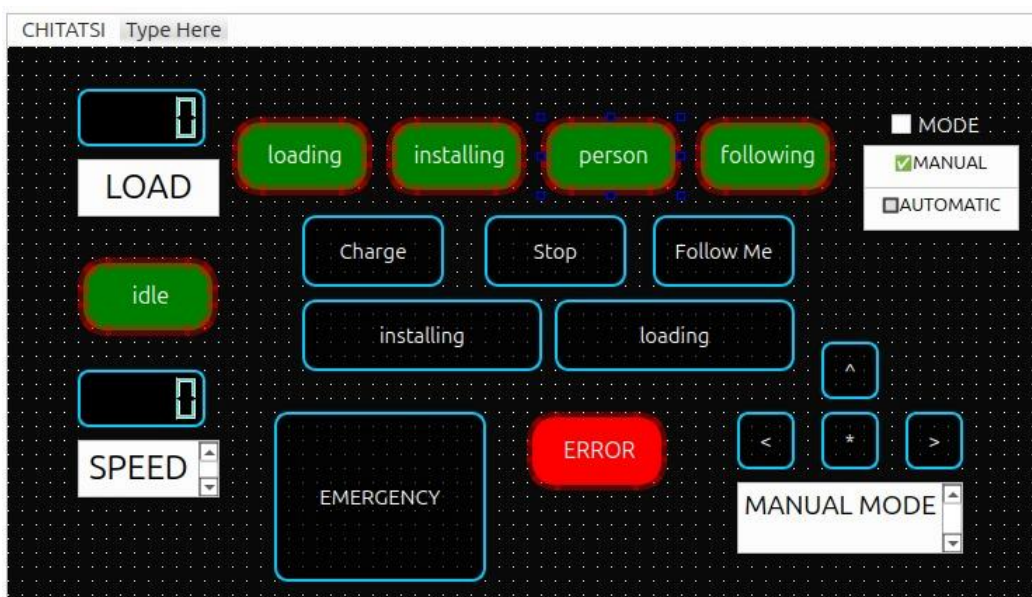


Figure 11— CHITATSI PC GUI

18 Mission Execution

Task completion. The robot performs the complete mission end-to-end: it is loaded with an item, follows the operator, navigates while avoiding static and dynamic obstacles, reaches the delivery point and releases the item through the gate.

Extra-mechanism functionality. The MG996R servo gate opens and closes reliably on command with a roughly four-times torque margin, securing the load during transport and releasing it on arrival.

System repeatability. The mission runs consistently across repeated trials: the state machine resets cleanly to IDLE between runs and the PID integrators reset at rest, preventing drift from accumulating and keeping behaviour reproducible.

Good subsystem integration quality. Vision, navigation, low-level motion control, the GUI and the extra mechanism all operate together over ROS 2 and micro-ROS as one coherent system, which is the clearest evidence that the individual subsystems were engineered to work as a whole.

19 Conclusion

CHITATSI was completed end-to-end, with each subsystem independently validated before integration. The vision module is trained on a custom dataset and publishes detections with confidence scores over ROS 2. The mechanical assembly matches the CAD within tolerance and includes an integrated servo gate, emergency stop and state indicators. The electrical system provides a hardware-level safety chain independent of firmware. Motion control uses a cascaded PID stack under FreeRTOS, with orientation from a gyro–encoder complementary filter, and micro-ROS bridges the ESP32 firmware into the ROS 2 graph.

Beyond the technical build, the market research confirms a financially feasible opportunity: a ~2.8-year payback and 36% annual return position the Follow-Me Warehouse Robot as a realistic, cost-effective assistant for the underserved mid-size e-commerce fulfilment segment in Egypt and MENA.

CHITATSI Team 4 — Sub 2A · MCT334 Design of Mechatronics Systems (2) · Ain Shams University, Faculty of Engineering · Spring 2026